```
count = 0
threads = 10.times.map do |i|
  Thread.new do
    sleep(rand(0.1))
    Thread.current[:mycount] = count
    count += 1
  end
end

threads.each do |t|
  t.join
  print t[:mycount], ", "
end
puts "count = #{count}"
```

produces:

```
7, 8, 5, 1, 6, 4, 0, 2, 9, 3, count = 10
```

The main thread waits for the subthreads to finish and then prints that thread's value of count. Just to make it interesting, we use rand(0.1) to have each thread wait a random of amount time before recording the value, so that we can't predict the order in which the threads will finish.

A subtle *race condition* exists in this code. A race condition occurs when two or more pieces of code (or hardware) both try to access some shared resource, and the outcome changes depending on the order in which they do so. In the example here, it is possible for one thread to set the value of its mycount variable to count, but before it gets a chance to increment count, the thread gets descheduled and another thread reuses the same value of count. These issues are fixed by synchronizing the access to shared resources such as the count variable, see Synchronization via Mutual Exclusion, on page 193.

Threads and Exceptions

If a thread raises an unhandled exception, what happens next depends on the setting of the Thread.abort_on_exception flag and on the setting of the interpreter's $DEBUG flag.

If abort_on_exception is false and the debug flag is not enabled (the default condition), an unhandled exception simply kills the current thread—all the rest continue to run. In fact, you don't even hear about the exception until you issue a join on the thread that raised it. In the following example, thread 1 blows up and fails to produce any output. However, you can still see the trace from the other threads:

```
threads = 4.times.map do |number|
  Thread.new(number) do |i|
    raise "Boom!" if i == 1
    print "#{i}\n"
  end
end
puts "Waiting"
sleep 0.1
puts "Done"
```

produces:

```
#<Thread:0x0000000102098820 prog.rb:2 run> terminated with exception
(report_on_exception is true):
```